

# BsC Devops

## Group D

Oriol Grau Moragues - s25137@itu.dk

Mohamed Karam Haybout - mhay@itu.dk

Student - xxxx@itu.dk

Jordan Cherry - s25121@itu.dk

Madeleine Jakobsen - majak@itu.dk

Tim Vogensen Hounsgaard - thou@itu.dk

Section Tim,

## System's perspective

## Process' perspective

### How do you monitor your systems and what precisely do you monitor? -Madeleine

The team has used a combination of Prometheus and Grafana to monitor the project. Prometheus is used to collect metrics, while Grafana is used to visualize the data into something that can easily be understood. We currently monitor the number of requests that certain API endpoints get, such as : the 'post' endpoint for messages. The Prometheus and Grafana systems were added with PBI #26.

### Brief description of how your security hardened your systems -Tim

The System has been hardened with a fire wall and a proxy server so all traffic coming to the web/api app goes through a proxy server. all communication between user and proxy, and proxy and apps are delivered through HTTPS using TLS encryption. We updated the docker images to use a hardened image for security, and to secure we not introducing new security vulnerabilities CodeQL and Docker Scout was put in place in the CI pipeline to sniff out vulnerabilities.

### How do you handle availability and scaling in your systems?

#### Docker Swarm and rolling upgrades

Docker Swarm was introduced to manage updates without taking the system offline by stopping and restarting the containers each time through docker-compose up. Swarm allowed for the updates to be applied incrementally, bringing up new containers with the updated images before stopping the old ones. This allowed for users and the simulator to have no downtime as it was deploying, which was particularly important to prevent simulator issues.

#### Staging

A staging branch was created to validate new changes before deployment, where there is no risk of affecting the production environment. The testing of these changes in a separate environment reduces the chance of needing to rollback on the main branch regarding integration issues.

#### Health check and rollback

After each deployment, the pipeline curls the web server image several times. If the service fails to respond, the pipeline automatically redeploys the previous commit's image, restoring the previous state automatically.

#### Database migration

The deploy workflow automatically detects if any EF Core migration files changed between the commits and applies them before bringing up new containers. This keeps the database schema in sync with the application.

#### Variable number of instances

By using infrastructure as code, we can provision new instances and add them to the system easily. As the user base grows, new servers can be spin up without additional work other than specifying the number of instances and re-

running the IaC tools. Furthermore, our load balancing setup reduces the chances of overloading specific web servers and the load balancers are monitored with keepalived, which switches the active load balancer in case the current one fails.

## **Reflection' perspective**

We decided to use the project files from the course BDSA as the starting point for this project, this meant that any existing errors in the previous project would be also present in this project. This led to the team getting many errors once the simulator began to run, specifically with the creation of users. The project we used indexed users in a very inefficient way, and meant that if two users were created at the same time, they could both share the same index, leading to users being overwritten in the database. The system could not handle asynchronous tasks.

The team fixed the issue by refactoring the way the user indexing worked, letting the database automatically assign indexes instead of doing it manually. We learned that it is important to have very in-depth testing, to see how well an application handles multiple requests and tasks at once. Stress testing can also be a good way to find failures in the system.

## **Smoke Deploy issues**

After we had implemented docker swarm and its rolling upgrade functionality, we encountered inconsistent issues regarding the smoke deploy step in our CICD. Occasionally a smoke deploy would hang on either docker swarm activity check, applying migrations, or rolling upgrade step.

We had made multiple fixes but the issue would keep resurfacing later. It was only after inspecting the staging server that we discovered the issue lied in the limited resources the server provides. Maxed out CPU and RAM made any other operation besides the already running docker containers impossible.

We managed to fix this by implementing a resource guard to free up the servers resources during a deployment but this solution unfortunately sacrifices the full uptime that we aimed to get via rolling upgrades.

Overall this entire endeavor showed us the importance of running workflows locally to validate their results instead of constantly committing code to see if it solves the problem. This approach should be aimed towards for the sake of maintainability and operation.

## **Use of Generative AI -Tim**

Generative AI has been used throughout the project to help us, complete the weekly tasks. AI has been used as a debug tool / sparring partner, when a person was stuck. AI has also been used to generate a starting point for task with new technologies. The group has their own preferences in AIs so Gemini, Chatgpt, Claude and Copilot has been used during the project All have been marked as co-author when used. Pro and cons follows in the use of AI. For our purpose it has been a great tool that has saved us countless hours searching on the web, however this "easy" way could also be a hinderance in the sense it might halluciate a solution to our problem that does not work, and us taking the "easy" made harder to catch that. All in all it can be good, but we have found limiting the amount of usage is beneficial.